

CO3 – AT-2

Policy Optimization Assessment

ASSESSMENT - 2

**Experiments – Actor-Critic Methods
(A2C, A3C) (Analyse – L4)**

N.PARNITA

192525322

CODE:

```
!pip install -q gymnasium[classic-control]

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import multiprocessing as mp

print("Libraries ready!")
class ActorCritic(nn.Module):
    def __init__(self):
        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(4, 128),
            nn.ReLU()
        )

        self.actor = nn.Linear(128, 2)
        self.critic = nn.Linear(128, 1)

    def forward(self, state):
        x = self.shared(state)
        return self.actor(x), self.critic(x)

gamma = 0.99
learning_rate = 0.001

print("Actor-Critic network created.")
def train_a2c(episodes=200):

    env = gym.make("CartPole-v1")
    model = ActorCritic()
    rewards = []

    for episode in range(episodes):

        state, _ = env.reset()
```

```

log_probs = []
values = []
episode_rewards = []
done = False

while not done:

    state_t = torch.FloatTensor(state).unsqueeze(0)

    logits, value = model(state_t)

    dist = torch.distributions.Categorical(logits=logits)
    action = dist.sample()

    next_state, reward, terminated, truncated, _ = \
        env.step(action.item())

    done = terminated or truncated

    log_probs.append(dist.log_prob(action))
    values.append(value.squeeze())
    episode_rewards.append(reward)

    state = next_state

# Discounted returns
returns = []
G = 0

for r in reversed(episode_rewards):
    G = r + gamma * G
    returns.insert(0, G)

returns = torch.tensor(returns, dtype=torch.float32)
values = torch.stack(values)
log_probs = torch.stack(log_probs)

advantage = returns - values.detach()

actor_loss = -(log_probs * advantage).mean()
critic_loss = (returns - values).pow(2).mean()

```

```

    loss = actor_loss + 0.5 * critic_loss

    model.zero_grad()
    loss.backward()

    with torch.no_grad():
        for p in model.parameters():
            p -= learning_rate * p.grad

    rewards.append(sum(episode_rewards))

env.close()
return model, rewards

a2c_model, a2c_rewards = train_a2c()

print("A2C training completed!")
def a3c_worker(global_model, result_queue, episodes):

    env = gym.make("CartPole-v1")

    local_model = ActorCritic()
    local_model.load_state_dict(global_model.state_dict())

    for _ in range(episodes):

        state, _ = env.reset()

        log_probs = []
        values = []
        rewards = []
        done = False

        while not done:

            state_t = torch.FloatTensor(state).unsqueeze(0)

            logits, value = local_model(state_t)

            dist = torch.distributions.Categorical(logits=logits)
            action = dist.sample()

```

```

next_state, reward, terminated, truncated, _ = \
    env.step(action.item())

done = terminated or truncated

log_probs.append(dist.log_prob(action))
values.append(value.squeeze())
rewards.append(reward)

state = next_state

# Calculate returns
returns = []
G = 0

for r in reversed(rewards):
    G = r + gamma * G
    returns.insert(0, G)

returns = torch.tensor(returns, dtype=torch.float32)

values = torch.stack(values)
log_probs = torch.stack(log_probs)

advantage = returns - values.detach()

actor_loss = -(log_probs * advantage).mean()
critic_loss = (returns - values).pow(2).mean()

loss = actor_loss + 0.5 * critic_loss

local_model.zero_grad()
loss.backward()

# Asynchronous update of shared global model
with torch.no_grad():
    for global_p, local_p in zip(
        global_model.parameters(),
        local_model.parameters()
    ):
        if local_p.grad is not None:

```

```

        global_p -= learning_rate * local_p.grad

    # Synchronize local worker with global model
    local_model.load_state_dict(global_model.state_dict())

    result_queue.put(sum(rewards))

env.close()

def train_a3c(workers=4, episodes_per_worker=50):

    context = mp.get_context("fork")

    global_model = ActorCritic()
    global_model.share_memory()

    result_queue = context.Queue()

    processes = []

    for _ in range(workers):

        p = context.Process(
            target=a3c_worker,
            args=(
                global_model,
                result_queue,
                episodes_per_worker
            )
        )

        p.start()
        processes.append(p)

    rewards = []

    total_episodes = workers * episodes_per_worker

    for _ in range(total_episodes):
        rewards.append(result_queue.get())

    for p in processes:

```

```

        p.join()

    return global_model, rewards

a3c_model, a3c_rewards = train_a3c(
    workers=4,
    episodes_per_worker=50
)

print("True A3C training completed!")
print("Workers used: 4")
print("Total A3C episodes:", len(a3c_rewards))

a2c_final = np.mean(a2c_rewards[-20:])
a3c_final = np.mean(a3c_rewards[-20:])

print("=" * 50)
print("CO3 - AT-2: ACTOR-CRITIC METHODS")
print("=" * 50)

print("A2C Final Average Reward :", round(a2c_final, 2))
print("A3C Final Average Reward :", round(a3c_final, 2))

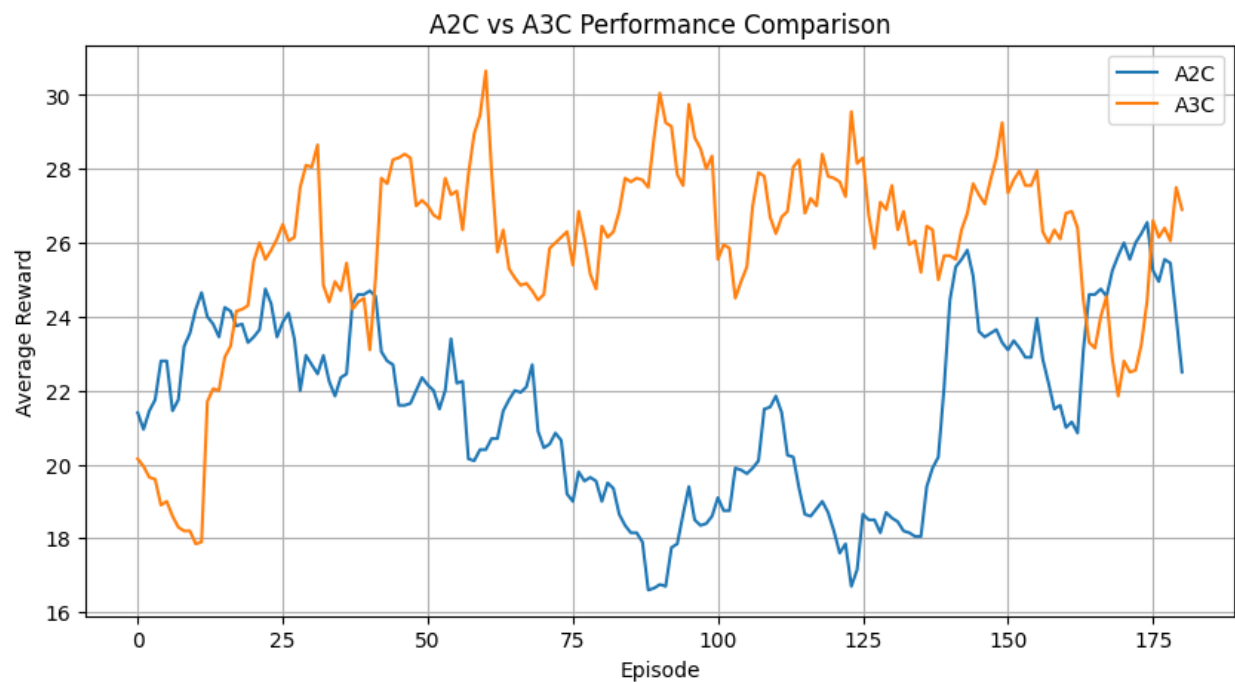
print("\nANALYSIS")
print("A2C performs synchronous policy updates.")
print("A3C uses multiple asynchronous workers.")

if a2c_final > a3c_final:
    print("A2C achieved the higher final reward.")
elif a3c_final > a2c_final:
    print("A3C achieved the higher final reward.")
else:
    print("Both methods achieved similar performance.")

```

OUTPUT:

Libraries ready!
Actor-Critic network created.
A2C training completed!
True A3C training completed!
Workers used: 4
Total A3C episodes: 200



=====

===

C03 - AT-2: ACTOR-CRITIC METHODS

=====

===

A2C Final Average Reward : 22.5

A3C Final Average Reward : 26.9

ANALYSIS

A2C performs synchronous policy updates.
A3C uses multiple asynchronous workers.
A3C achieved the higher final reward.